

Journal Pre-proof

Benders decompositions for order acceptance and scheduling in additive manufacturing

Jian Chen , Xudong Ye , Naiming Xie , Yao-Wen Sang ,
Malgorzata Sterna

PII: S0377-2217(25)00806-9
DOI: <https://doi.org/10.1016/j.ejor.2025.10.005>
Reference: EOR 19806



To appear in: *European Journal of Operational Research*

Received date: 13 September 2024

Accepted date: 4 October 2025

Please cite this article as: Jian Chen , Xudong Ye , Naiming Xie , Yao-Wen Sang ,
Malgorzata Sterna , Benders decompositions for order acceptance and scheduling
in additive manufacturing, *European Journal of Operational Research* (2025), doi:
<https://doi.org/10.1016/j.ejor.2025.10.005>

This is a PDF file of an article that has undergone enhancements after acceptance, such as the addition of a cover page and metadata, and formatting for readability, but it is not yet the definitive version of record. This version will undergo additional copyediting, typesetting and review before it is published in its final form, but we are providing this version to give early visibility of the article. Please note that, during the production process, errors may be discovered which could affect the content, and all legal disclaimers that apply to the journal pertain.

© 2025 Published by Elsevier B.V.

Highlights

- This study investigates a novel scheduling problem in additive manufacturing that integrates order acceptance.
- Two MIP models are formulated to jointly optimize order acceptance, build formation and schedule construction.
- A logic-based and a nested logic-based Benders decomposition framework are developed.
- Several Benders cuts and search strategies are employed to enhance solution efficiency.
- Extensive experiments demonstrate that NLBBD outperforms MIP and LBBD in both quality and speed.

Benders decompositions for order acceptance and scheduling in additive manufacturing

Jian Chen ^a, Xudong Ye ^a, Naiming Xie ^a, Yao-Wen Sang ^{a,*}, Malgorzata Sterna ^b

^a College of Economics and Management, Nanjing University of Aeronautics and Astronautics,
Nanjing, Jiangsu 210016, PR China

^b Institute of Computing Science, Poznan University of Technology,
Piotrowo 2, Poznan 60-965, Poland

Abstract

We study an order acceptance and parallel machine scheduling problem appearing in additive manufacturing (AM) considering two-dimensional packing with part rotation. Three joint decisions, including order acceptance, build formation and schedule construction, are made to maximize the revenue of accepted orders minus a weighted makespan, thereby capturing the trade-off between profitability and production efficiency. First, two mixed integer programming models (MIP) are formulated to optimize the three mentioned decisions simultaneously. Then, due to the specific structure of the problem, we propose a general logic-based Benders decomposition (LBBD) approach and a nested logic-based Benders decomposition (NLBBD) approach with various Benders cuts and search strategies. Finally, we undertake a comprehensive computational study to validate the effectiveness of all the proposed approaches. The computational results show that NLBBD achieves 50% more optimal solutions than MIP models implemented in Gurobi and 35% more than LBBD for small-sized instances. For large-sized instances, NLBBD still achieves better solutions with smaller gaps compared to Gurobi and LBBD. Further, numerical experiments confirm the effectiveness of the designed additional cuts and search strategies applied in Benders decompositions.

Keywords: Scheduling, Order acceptance, Additive manufacturing, Two-dimensional packing, Nested logic-based Benders decomposition

*Corresponding author

Email address: ywsang@nuaa.edu.cn (Yao-Wen Sang)

1. Introduction

Additive manufacturing (AM), more commonly known as three-dimensional (3D) printing, has become a pivotal component in the fabric of Industry 4.0, which is the current industrial revolution characterized by the fusion of digital technologies with traditional manufacturing processes. AM technology enables the creation of objects by adding material layer by layer, thereby offering a high degree of design flexibility and the ability to produce complex geometries that are not achievable with conventional subtractive manufacturing methods. Industries ranging from aerospace and automotive to healthcare are adopting AM to produce customized parts in smaller batches, which traditional manufacturing methods struggle to handle efficiently (Attaran, 2017). However, despite its advantages, AM remains a costly process, not only due to the high expense of raw materials but also because of its relatively slow production rate. Hence, it is important to optimize the production scheduling in order to get the maximum output possible.

In many real-world AM scenarios, production planning involves three decisions (as shown in Figure 1), i.e., selecting which part orders to accept given limited AM machine capacity (order acceptance), packing accepted parts into 2D build platforms without overlapping (build formation), and scheduling these builds across multiple machines (schedule construction). Each part is associated with a revenue, and naturally, a manufacturer hopes to accept as many parts as possible to maximize total revenue. However, due to limited capacity, accepting too many parts may prolong the makespan and compromise service quality, making strategic order acceptance essential. For accepted parts, they must be packed into build platforms. Each build platform can contain multiple parts, but all parts within the same build platform must be arranged without geometric overlap, satisfying 2D constraints. In the scheduling stage, builds should be assigned to a set of available AM machines for processing. During the processing stage, a thin layer of metal powder is repeatedly spread across the build platform, and a high-power laser or electron beam selectively melts the powder based

on the digital CAD design, forming solid layers. This process continues layer by layer until all parts in the build are completed. The processing time of a build depends on the total volume of the parts in this build and their maximum height. Effective minimization of the system-wide makespan requires careful consideration of each build's processing time and a balanced assignment of builds across machines. In summary, order acceptance determines revenue potential, while build formation and schedule construction affect production efficiency. A manufacturer naturally values both revenue generation and production efficiency, and typically holds a preference between the two. Accordingly, the objective is to maximize the total revenue from accepted parts minus a weighted makespan penalty, with the weight capturing the manufacturer's trade-off.

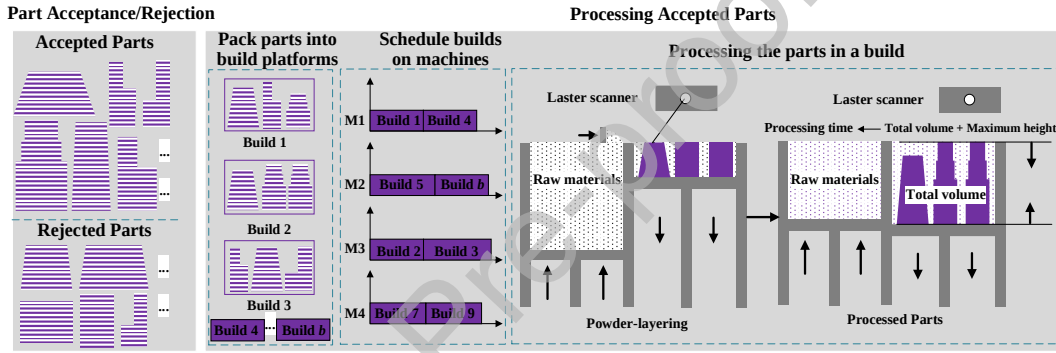


Figure 1. The processing of parts in AM

Motivated by the above scenario, we investigate an order acceptance and parallel machine scheduling problem in AM considering 2D packing and part rotation, referred to as OAPMSAM-2DPR. The problem involves the decisions of order acceptance, build formation, and schedule construction, aiming to maximize the total revenue minus the weighted makespan. Due to the limited capacity of AM machines and varying part revenues, manufacturers must carefully select parts to maximize revenue (Kalantari et al., 2011; Wang et al., 2019). Unlike batch processing machine scheduling problems, the build formation of AM scheduling needs to make an additional strategic decision, i.e., determining how to pack parts into build platforms such that no overlap occurs among parts within the same build (Li et al., 2017). Build formation is critical because efficient packing, while satisfying 2D constraints, can increase build density and reduce the number of builds required to pack all parts, thus improving the production efficiency. Furthermore, balanced scheduling of builds across machines is essential for minimizing

the makespan and enhancing machine utilization (Pinedo, 2012).

In this work, we first propose two mixed integer programming models (MIP) for OAPMSAM-2DPR problem, and develop a general logic-based Benders decomposition (LBBD) approach and a nested logic-based Benders decomposition (NLBBD) approach with various Benders cuts and search strategies (also referred to as acceleration strategies in some studies). The LBBD method decomposes the problem into a master problem, which determines order acceptance, build formation without 2D constraints, and schedule construction, and a set of subproblems that verify the 2D constraints for each build. Based on this structure, the NLBBD approach introduces an additional layer of decomposition within the master problem, dividing the problem into one master problem and two levels of subproblems. Moreover, additional Benders cuts and search strategies are employed to enhance the algorithms' efficiency. The effectiveness of MIP models and both decomposition methods, as well as the Benders cuts and search strategies is demonstrated through extensive numerical experiments.

The structure of the paper is as follows. Section 2 reviews more deeply the literature closely related to our research. Section 3 introduces the formal definition of the considered problem and presents two MIP models. Section 4 proposes the LBBD approach and the NLBBD approach. Section 5 reports results of numerical experiments conducted for the proposed models and algorithms. Finally, Section 6 provides conclusions and outlines future research directions.

2. Literature review

In this section, we conduct a focused review of the literature most relevant to the OAPMSAM-2DPR problem introduced in the Introduction, i.e., concerning two main research streams on scheduling with order acceptance and scheduling in additive manufacturing. Then, the research gap resulting from this survey is pointed out. Moreover, we shortly present the literature related to Benders decomposition and its applications.

2.1 Order acceptance and scheduling

The order acceptance and scheduling (OAS) problem is a concept that focuses on

the integrated optimization of two decisions on accepting orders and scheduling them (Slotnick, 2011). This problem can also be regarded as a scheduling problem with rejection (Shabtay, et al., 2013).

A significant portion of OAS research has primarily concentrated on the single machine environment (Geramipour et al., 2017; Nobibon and Leus, 2011; Zhang et al., 2009; Tarhan and Oğuz, 2021). Slotnick and Morton (1996; 2007) introduced branch-and-bound algorithms for solving the OAS problem on a single machine, with the objective of maximizing revenue while minimizing tardiness costs. Additionally, Rom and Slotnick (2009) explored the application of genetic algorithms to OAS problems and provided valuable insights for future research in this area.

Other researchers expanded OAS to parallel machine environments. The OAS problem on identical parallel machines was initially explored by Bartal et al. (2000), where jobs could be rejected with a specified penalty. Liu and Lu (2020) presented an approximation algorithm to solve the identical machine scheduling problem with job rejection. Naderi and Roshanaei (2020) proposed a branch-relax-and-check (BRC) algorithm to address an order acceptance and identical parallel machine scheduling problem. Emami et al. (2016; 2017) studied the OAS problem within the context of unrelated parallel machines, taking into consideration uncertain revenue and processing times. They devised a robust counterpart to the MILP model, employing a Benders decomposition approach for constructing solutions. Chen et al. (2024) investigated an integrated OAS problem for heterogeneous factories with carbon caps. The LBB method combined with branch-and-check framework was proposed to address the problem. Sang et al. (2025) considered the scheduling problem for minimizing the total weighted late work and rejection cost. They provided a pseudo-polynomial time algorithm and a fully polynomial time approximation scheme for the Pareto-scheduling problem. In addition, some of the literature combined the OAS problem with logistics decisions (Tarhan and Oğuz, 2021; Xue and Wang, 2023).

2.2 Scheduling in additive manufacturing

The scheduling problems arising in AM can be described as packing parts into build platforms while guaranteeing that the parts in the same build platform are placed without overlapping, and assigning and scheduling the builds on the machines. During this optimization process, it is important to ensure that the physical capacity of AM machine is sufficient to hold all allocated parts. In the remainder of this section, we review previous AM scheduling studies and summarize them in Table 1.

This issue was initially outlined by Li et al. (2017), who introduced a mathematical model alongside two heuristic approaches. Their objective was to minimize the average production cost per unit volume of material. Fera et al. (2018) studied a single-machine scheduling problem in AM considering two objectives: total earliness-tardiness and production costs. A multi-objective mathematical model and a genetic algorithm were proposed for this problem. Kucukkoc (2019) built various MIP models for scheduling single and multiple parallel AM machines with the objective of minimizing makespan. Alicastro et al. (2021) proposed a reinforcement learning iterated local search meta-heuristic based on the implementation of a Q-Learning variable neighborhood search to address an identical parallel machine scheduling problem in AM. Ying et al. (2022) extended the iterative greedy algorithm to solve the AM scheduling problem in a single machine production environment. Zipfel et al. (2024) studied a part scheduling problem for unrelated parallel AM machines, aiming to minimize the total weighted tardiness of all customer orders. To solve the problem, a MIP model and an iterated local search algorithm were developed. However, most of these studies address a one-dimensional constraint during grouping parts into builds, which ensures that the total area of the parts assigned to the same build does not exceed the build platform's area of the AM machine. The scheduling problem with 2D constraints in AM shares certain similarities with 2D batch scheduling problems. A fundamental distinction lies in how the processing time of a batch is defined. In 2D batch scheduling problems, the batch processing time is typically determined by the longest processing time among the jobs in the batch (Li and Zhang, 2018; Zhou et al., 2022; Hu et al., 2024). In AM, the batch

processing time depends on both the total volume of all parts in the batch and the maximum height among those parts. This results in stronger interplay among order acceptance, build formation, and schedule construction, making the modeling and solution process more complex than 2D batch scheduling problems.

Due to the heightened complexity introduced by taking into account more realistic 2D constraints, most studies resort to employing non-exact methods to tackle such problems. Aloui and Hadj-Hamou (2021) focused on the scheduling and nesting problem in AM with technological constraints to minimize the total delay of parts and maximize the use rate of machines. A heuristic approach was developed to solve large-sized instances. Hu et al. (2022) proposed an adaptive large neighborhood search (ALNS) to address an unrelated parallel machine scheduling problem in AM with two practical constraints: 2D constraints and unequal part release times. Lee and Kim (2023) studied a scheduling problem in parallel AM considering 2D placement and 3D rotation and proposed a two-stage meta-heuristic method. Hu et al. (2024) built upon their previous research (Hu et al. 2022) by developing a novel ALNS heuristic, integrating innovative destroy and repair operators to enhance the method's efficacy.

Only the following papers align closely with our study, all of which utilize exact algorithms to address the scheduling problem in AM considering 2D constraints. Liu et al. (2023) provided a branch-and-price algorithm to solve the scheduling problem in AM considering a single machine. Mao et al. (2024) proposed an exact algorithm based on combinatorial Benders decomposition (CBD) method with various Benders cuts to address a single-machine scheduling problem in AM. Nascimento et al. (2024) studied a parallel machine scheduling problem in AM considering irregular-shaped parts with specific release dates, processing times, and due dates. To address this problem, they developed a LBBD approach. The first two papers focused on scheduling problems involving a single AM machine, while the third addressed the processing of irregular parts without considering part rotation. In contrast, our work investigates OAS on parallel AM machines, incorporating 2D constraints with part rotation. Therefore, to

the best of our knowledge, no studies similar to ours have been conducted so far.

Journal Pre-proof

Table 1. Classification of AM scheduling problems

Reference	Order acceptance	Packing			Part rotation	Part shape		Machines configuration			Objective function	Solving approach
		1D	2D	3D		Rect.	Irr.	Single	Ident.	Unrel.		
Fera et al. (2018)		✓						✓			Total earliness-tardiness, costs	GA
Kucukkoc (2019)		✓						✓	✓	✓	Makespan	MILP
Alicastro et al. (2021)		✓							✓		Makespan	Reinforcement Learning ILS
Aloui and Hadj-Hamou (2021)			✓			✓			✓		Total delay, use rate	Heuristic
Ying et al. (2022)		✓						✓			Total earliness-tardiness, costs	Iterative greedy algorithm
Hu et al. (2022)			✓		✓	✓				✓	Makespan	ALNS
Liu et al. (2023)			✓			✓		✓			Makespan	Branch-and-price
Lee and Kim (2023)				✓	✓	✓			✓		Makespan	Two-stage GA and PSO
Hu et al. (2024)			✓		✓	✓				✓	Makespan	Novel ALNS
Zipfel et al. (2024)		✓								✓	Total weighted tardiness	ILS
Mao et al. (2024)			✓			✓		✓			Makespan	CBD
Nascimento et al. (2024)			✓				✓		✓		Tardiness	LBBD
This paper	✓		✓		✓	✓			✓		Profit	NLBBD

Rect. – Considering rectangular bounding boxes; Irr. – Considering irregular parts; Ident. – Identical Parallel Machines; Unrel. – Unrelated Parallel Machines.

2.3 Benders decomposition and its applications

Benders decomposition method (Benders, 1962) is an exact algorithm designed for efficiently solving MIP models. This approach partitions the original problem into a relaxed master problem and a linear subproblem. In the Benders decomposition, an iterative process occurs between the master problem and the subproblems. Information flows from the master problem to the subproblems by solving the former, while the latter contributes Benders cuts to the master problem. This iterative process continues until the objective values of the master problem and subproblem match, at which point the current solution is confirmed as optimal. Hooker and Ottosson (2003) proposed the LBBD method based on the Benders decomposition, which can generate cuts based on the logical relationship between the master problem and the subproblems, and applied it to scheduling (Hooker, 2007). In the following, we briefly review the successful applications of Benders decomposition on scheduling area.

Tran et al. (2016) investigated the unrelated parallel machine scheduling problem, incorporating sequence and machine-dependent setup times, with the aim of minimizing makespan. They developed two exact decomposition-based methods using LBBD and a branch-and-check. Their computational results demonstrated that the proposed methods outperformed existing MIP models in both speed and scalability. Chen et al. (2023) devised an LBBD method coupled with a branch-and-check search framework to tackle an OAS problem in distributed manufacturing. Elçi and Hooker (2022) developed a LBBD method to address two-stage stochastic planning and scheduling problems. Li et al. (2023) tackled an order assignment and scheduling problem with direct distribution under uncertainties in processing and distribution times. The applications of Benders decomposition method can be found in collaborative operating room planning and scheduling (Roshanaei et al., 2017a; 2017b; 2020), bin packing problem (Côté et al., 2021), plant location problem (Fazel-Zarandi and Beck, 2012) and supply chain network design (Naderi et al., 2020).

3. Problem description and mathematical formulation

In this section, we first formally define the order acceptance and parallel machine scheduling problem in AM considering 2D packing and part rotation and next we build two different mixed integer programming models (MIP) for this problem.

3.1 Problem description

Given a set of parts $P = \{1, \dots, p, \dots, |P|\}$, representing customer orders, and a set of identical parallel AM machines $M = \{1, \dots, m, \dots, |M|\}$. The goal is to accept some parts for production, to pack these accepted parts on builds $B = \{1, \dots, b, \dots, |B|\}$, and process these builds on machines for maximizing the profit.

Each part is associated with its revenue, definite volume and shape. The revenue of part p is U_p in case of acceptance and 0 in case of rejection. The volume of part p is denoted by v_p . In general, the shape of part p might be irregular, but similar to Lee and Kim (2023), we model it as the smallest cuboid that can contain it, defined by length l_p , width w_p , and height h_p . Obviously, the volume of a part cannot be greater than the volume of its corresponding cuboid, i.e., $v_p \leq l_p \times w_p \times h_p$. The number of builds is defined as $|B| = |P|$, representing the worst case in which each part occupies a separate build. Note that the actual number of builds used is determined by the optimization process and does not exceed $|P|$. All builds are two dimensional and have identical size, with length L and width W . When packing multiple parts into a build, the following conditions must be met: (1) parts cannot exceed the 2D-build-space $L \times W$; (2) parts in the same build cannot overlap; (3) parts can be rotated around Z-axis 90 degrees.

The processing time bt_b of build b is determined by its total volume V_b and the maximum height H_b among the parts within this build. The total volume V_b is defined as $V_b = \sum_{p \in P} v_p \times z_{pb}$, where $z_{pb} = 1$ if part p is packed in build b and $z_{pb} = 0$ otherwise. The maximum height is defined as $H_b = \max\{z_{pb} \times h_p : p \in P\}$. Let SET , HT and VT be the setup time, the time spent per powder-layering, and the time spent by a machine per unit volume, respectively. Hence, the processing time of build b is $bt_b = SET + HT \times H_b + VT \times V_b$.

The dual concern for revenue generation and production efficiency motivates the formulation of the objective function in this study, which aims to maximize the total

revenue from accepted parts minus a weighted makespan penalty. Specifically, the total revenue is expressed as $\sum_{p \in P} (Y_p - 1)U_p$, where Y_p is a binary decision variable indicating whether part p is rejected ($Y_p = 1$) or accepted ($Y_p = 0$), and U_p denotes the revenue associated with part p . The makespan penalty is represented by $\alpha \cdot MS$, where MS denotes the system-wide makespan, i.e., the maximum completion time across all parallel machines processing the accepted parts, and α is a weight parameter reflecting the manufacturer's valuation of time-based operational costs relative to revenue gains. To ensure that both revenue and makespan have a meaningful impact on the objective function, in this paper parameter α is defined such that the revenue and makespan terms are on the same scale, or at least on a comparable scale.

Makespan minimization is a fundamental objective in scheduling; however, it becomes challenging in the OAPMSAM-2DPR problem due to the need to coordinate three decisions, i.e., order acceptance (which may prioritize parts with high revenue-to-size ratios), build formation (which determines the total processing workloads), and schedule construction (which affects the balance of workloads across machines), as illustrated in Figure 2. The parameter α serves as a tunable control for decision-makers, enabling them to balance aggressive order intake with the need to limit production duration. This unified objective function thus captures the inherent tension between the total revenue and the production efficiency, providing a flexible and practical framework for preference-aware decision-making in AM environments. For a more detailed understanding of the OAPMSAM-2DPR problem, an illustrative example is provided in Appendix A.

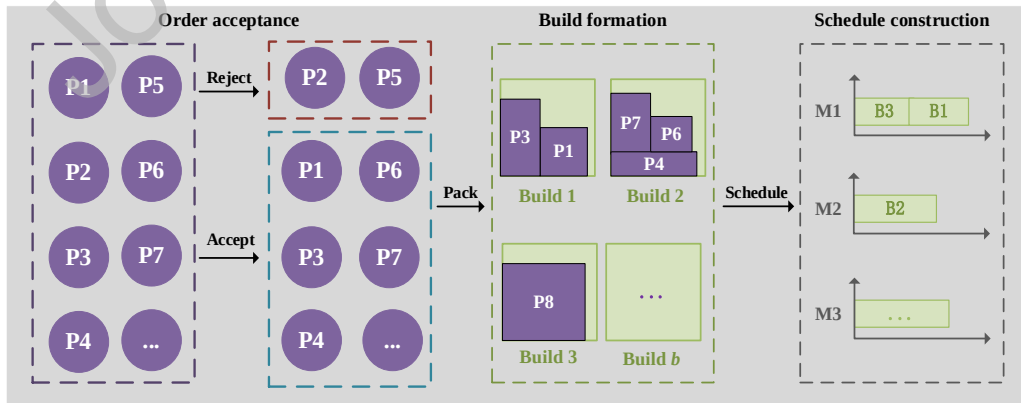


Figure 2. Schematic chart of the OAPMSAM-2DPR problem

3.2 Mathematical models

To strictly formulate the optimization problem presented above we propose two mathematical integer programming models. Model 1 considers the packing of parts into builds and the scheduling of these builds as two separate decisions, each represented by distinct sets of constraints and decision variables. While Model 2 integrates these two decisions into a cohesive framework. The notations of sets, parameters and decision variables, as well as auxiliary variables necessary to build a model are as follows.

Sets:

- P : set of parts
 B : set of builds
 M : set of machines

Parameters:

- U_p : revenue of accepted part p
 w_p : width of part p
 l_p : length of part p
 h_p : height of part p
 v_p : volume of part p
 W : width of a build
 L : length of a build
 SET : machine setup time
 VT : time spent by a machine per unit volume
 HT : time spent per powder-layering
 α : tunable control parameter that reflects the decision-maker's preference in balancing revenue against makespan
 BM : large integer

Decision variables and auxiliary variables determined by decision variables:

- MS : makespan
 x_p : the X-axis coordinate of the bottom-left point of part p
 y_p : the Y-axis coordinate of the bottom-left point of part p
 L_p : the size of part p considered as its length after assigning to the build
 W_p : the size of part p considered as its width after assigning to the build
 H_b : height of build b
 V_b : volume of build b

- s_b : starting time of build b
 bt_b : processing time of build b
 Y_p : 1 if part p is rejected, 0 otherwise
 z_{pb} : 1 if part p is packed in build b , 0 otherwise
 I_p^W : 1 if the width of part p is assigned to the width of the build in which it is put and the length of part p is assigned to the length of this build, 0 otherwise
 I_p^L : 1 if the width of part p is assigned to the length of the build in which it is put and the length of part p is assigned to the width of this build, 0 otherwise
 o_{jk}^X : 1 if part $j \in P$ precedes part $k \in P$ on X-axis when they are packed in the same build, 0 otherwise
 o_{jk}^Y : 1 if part $j \in P$ is positioned below part $k \in P$ on Y-axis when they are packed in the same build, 0 otherwise
 A_{bm} : 1 if build $b \in B$ is assigned to machine $m \in M$, 0 otherwise
 f_{ab} : 1 if build $a \in B$ precedes build $b \in B$ when they are assigned to the same machine, 0 otherwise

The first mathematical integer programming model of the considered problem is formulated as follows.

(Model 1)

$$\max \sum_{p \in P} U_p(1 - Y_p) - \alpha \cdot MS$$

s. t.

$$\sum_{b \in B} z_{pb} + Y_p = 1 \quad \forall p \in P \quad (1)$$

$$I_p^W + I_p^L = 1 \quad \forall p \in P \quad (2)$$

$$L_p = I_p^W \times l_p + I_p^L \times w_p \quad \forall p \in P \quad (3)$$

$$W_p = I_p^W \times w_p + I_p^L \times l_p \quad \forall p \in P \quad (4)$$

$$x_p + L_p \leq L \quad \forall p \in P \quad (5)$$

$$y_p + W_p \leq W \quad \forall p \in P \quad (6)$$

$$x_j + L_j \leq x_k + BM \times (3 - o_{jk}^X - z_{jb} - z_{kb}) \quad \forall j, k \in P, j \neq k, b \in B \quad (7)$$

$$o_{jk}^X + o_{kj}^X \leq 1 \quad \forall j, k \in P, j \neq k \quad (8)$$

$$y_j + w_j \leq y_k + BM \times (3 - o_{jk}^X - z_{jb} - z_{kb}) \quad \forall j, k \in P, j \neq k, b \in B \quad (9)$$

$$o_{jk}^Y + o_{kj}^Y \leq 1 \quad \forall j, k \in P, j \neq k \quad (10)$$

$$o_{jk}^X + o_{kj}^X + o_{jk}^Y + o_{kj}^Y - z_{jb} - z_{kb} \geq -1 \quad \forall j, k \in P, j \neq k, b \in B \quad (11)$$

$$\sum_{p \in P} z_{p(b+1)} \leq BM \times \sum_{p \in P} z_{pb} \quad \forall b \in B \quad (12)$$

$$V_b = \sum_{p \in P} v_p \times z_{pb} \quad \forall b \in B \quad (13)$$

$$H_b \geq BM \times (z_{pb} - 1) + h_p \quad \forall p \in P, b \in B \quad (14)$$

$$bt_b \geq SET \times z_{pb} + HT \times H_b + VT \times V_b \quad \forall p \in P, b \in B \quad (15)$$

$$\sum_{m \in M} A_{bm} = 1 \quad \forall b \in B \quad (16)$$

$$s_a + bt_a \leq s_b + BM \times (3 - f_{ab} - A_{am} - A_{bm}) \quad \forall a, b \in B, a < b, m \in M \quad (17)$$

$$s_b + bt_b \leq s_a + BM \times (2 + f_{ab} - A_{am} - A_{bm}) \quad \forall a, b \in B, a < b, m \in M \quad (18)$$

$$MS \geq s_b + bt_b \quad \forall b \in B \quad (19)$$

$$V_b, H_b, s_b, bt_b \geq 0 \quad \forall b \in B \quad (20)$$

$$x_p, y_p, L_p, W_p \geq 0 \quad \forall p \in P \quad (21)$$

$$Y_p, I_p^W, I_p^L \in \{0,1\} \quad \forall p \in P \quad (22)$$

$$o_{jk}^X, o_{jk}^Y \in \{0,1\} \quad \forall j, k \in P, j \neq k \quad (23)$$

$$z_{pb} \in \{0,1\} \quad \forall p \in P, b \in B \quad (24)$$

$$f_{ab} \in \{0,1\} \quad \forall a, b \in B, a < b \quad (25)$$

$$A_{bm} \in \{0,1\} \quad \forall b \in B, m \in M \quad (26)$$

The objective function is to maximize the revenue of the accepted parts decreased by the weighted makespan penalty. In fact, since the total potential revenue of all parts is fixed, maximizing the revenue of the accepted parts is equivalent to minimizing the revenue loss due to the rejected parts. Accordingly, the objective function can also be reformulated as minimizing the revenue loss of the rejected parts plus the weighted makespan penalty. Moreover, an objective of the form $X + \alpha Y$ can be normalized into a classical weighted-sum formulation by dividing by $1 + \alpha$, giving $X/(1 + \alpha) + \alpha Y/(1 + \alpha)$, where the coefficients lie in $[0,1]$ and sum to 1. Constraints (1) select parts and pack the accepted parts into a build. Constraints (2) determine the orientation of each part. Constraints (3) and (4) determine the size (dimension) of part p assigned to the length and width of a build. We establish a right-angle coordinate system with

the lower left corner of the build as the origin. The X-axis represents the length of the build, while the Y-axis represents its width. Constraints (5) and (6) ensure that the parts are fully placed within the build space. Constraints (7)-(11) define the positional relationships between two parts placed within the same build (see Appendix B for detailed explanations). Constraints (12) guarantee that builds are utilized in an incremental order starting from build 1. Constraints (13)-(15) calculate the volume, the height and the processing time of a build based on ideas proposed by Kucukkoc (2019). Constraints (16) ensure that one build is scheduled only on one machine. Constraints (17) and (18) feasibly sequence the builds on each machine and compute the start time of these builds. Constraints (19) calculate the makespan of AM production. Constraints (20)-(26) define the domains of the decision variables.

To propose another mathematical integer programming model for OAPMSAM-2DPR problem, we replace two decision variables A_{bm} and z_{pb} with a new single decision variable A_{mbp} . As a consequence, to the variables already defined in Model 1, we add variables given below together with appropriate constraints, and we remove variables z_{pb} , A_{bm} and f_{ab} , as well as constraints related to them.

Decision variables and auxiliary variables determined by decision variables:

H_{mb} : height of build b on machine m

V_{mb} : volume of build b on machine m

c_{mb} : completion time of build b on machine m

bt_{mb} : processing time of build b on machine m

A_{mbp} : 1 if part p is assigned to build b on machine m , 0 otherwise

The alternative model of the original problem can be formulated as follows:

(Model 2)

$$\max \sum_{p \in P} U_p(1 - Y_p) - \alpha \cdot MS$$

s. t.

$$\sum_{m \in M} \sum_{b \in B} A_{mbp} + Y_p = 1 \quad \forall p \in P \quad (27)$$

Constraints (2) ~ (6)

$$x_j + L_j \leq x_k + BM \times (3 - o_{jk}^X - A_{mbj} - A_{mbk}) \quad \forall j, k \in P, j \neq k, b \in B, m \in M \quad (28)$$

$$o_{jk}^X + o_{kj}^X \leq 1 \quad \forall j, k \in P, j \neq k \quad (29)$$

$$y_j + w_j \leq y_k + BM \times (3 - o_{jk}^Y - A_{mbj} - A_{mbk}) \quad \forall j, k \in P, j \neq k, b \in B, m \in M \quad (30)$$

$$o_{jk}^Y + o_{kj}^Y \leq 1 \quad \forall j, k \in P, j \neq k \quad (31)$$

$$o_{jk}^X + o_{kj}^X + o_{jk}^Y + o_{kj}^Y - A_{mbj} - A_{mbk} \geq -1 \quad \forall j, k \in P, j \neq k, b \in B, m \in M \quad (32)$$

$$V_{mb} = \sum_{p \in P} v_p \times A_{mbp} \quad \forall b \in B, m \in M \quad (33)$$

$$H_{mb} \geq BM \times (A_{mbp} - 1) + h_p \quad \forall p \in P, b \in B, m \in M \quad (34)$$

$$bt_{mb} \geq SET \times A_{mbp} + HT \times H_{mb} + VT \times V_{mb} \quad \forall p \in P, b \in B, m \in M \quad (35)$$

$$\sum_{p \in P} A_{m(b+1)p} \leq BM \times \sum_{p \in P} A_{mbp} \quad \forall b \in B, m \in M \quad (36)$$

$$c_{m(b-1)} + bt_{mb} \leq c_{mb} \quad \forall b \in B, m \in M \quad (37)$$

$$c_{m0} = 0 \quad \forall m \in M \quad (38)$$

$$MS \geq c_{mb} \quad \forall b \in B, m \in M \quad (39)$$

Constraints (21) ~ (23)

$$V_{mb}, H_{mb}, c_{mb}, bt_{mb} \geq 0 \quad \forall b \in B, m \in M \quad (40)$$

$$A_{mbp} \in \{0,1\} \quad \forall b \in B, m \in M, p \in P \quad (41)$$

Constraints (27) select parts and pack the accepted parts into a build on machine m . Constraints (28) and (29) determine the positional relationship of two parts packed in the same build on the X-axis. Constraints (30) and (31) follow the logic of Constraints (28) and (29) on the Y-axis. Constraints (32) determine the positional relationship of two parts in the build both on the X and Y-axis. Constraints (33)-(35) calculate the volume, the height and the processing time of a build on machine m . Constraints (36) guarantee that builds on machine m are utilized in an incremental order starting from build 1. Constraints (37) and (38) compute the completion times of the builds on machine m . Constraints (39) calculate the makespan. Constraints (40) and (41) define the domains of the new decision variables.

4. Solution methods

To solve the OAPMSAM-2DPR problem, we propose two exact decomposition algorithms, i.e., a general LBB and a NLBB. These methods are motivated by the fact that the OAPMSAM-2DPR problem tightly integrates order acceptance, build formation with 2D constraints, and schedule construction across parallel machines,

posing challenges for standard methods. MIP-based approaches suffer from scalability issues due to the combinatorial explosion of decision variables, particularly when incorporating 2D constraints. Moreover, classical decomposition techniques, such as standard Benders decomposition, rely on LP relaxations and are not suited to capture the logical dependencies across interrelated subproblems. In contrast, the LBBD framework leverages the problem's logical structure by decoupling the overall problem into modular components and allowing for tailored solution approaches. However, when master problems become increasingly complex due to embedded order acceptance, build formation excluding 2D constraints, and scheduling construction, even the LBBD can become inefficient. To address this, the NLBBD method introduces an additional layer of decomposition within the master problem, enabling a two-level framework that further exploits the problem's nested structure. This nested approach enhances computational tractability while maintaining solution quality, making it well-suited for solving the highly integrated OAPMSAM-2DPR problem.

4.1 Logic-based Benders decomposition

Based on the LBBD originally proposed by Hooker and Ottosson (2003), and its successful applications to scheduling problems as demonstrated by Tran et al. (2016), Chen et al. (2023), and Elçi and Hooker (2022), we further adapt the LBBD method to address the OAPMSAM-2DPR problem. In this paper, the proposed LBBD method decomposes the original problem into a master problem (LBBD-MP) of order acceptance, build formation not taking into account 2D constraints, and schedule construction, as well as multiple subproblems (LBBD-SP_{*i*}) based on the master problem solution to determine whether the parts grouped to build *i* can satisfy its 2D constraints. If an infeasible part allocation is identified through the subproblem solution, the master problem generates corresponding Benders cuts. This iterative process continues until an optimal solution is attained. More details are given below.

4.1.1 Master problem of the LBBD

The master problem in LBBD, denoted as LBBD-MP, focuses on determining decisions on order acceptance, allocating parts to builds, but excluding 2D constraints, and on scheduling builds on AM machines.

(LBBD-MP)

$$\max \sum_{p \in P} U_p(1 - Y_p) - \alpha \cdot MS$$

s. t.

$$\sum_{p \in P} A_{mbp} \times l_p \times w_p \leq L \times W \quad \forall b \in B, m \in M \quad (42)$$

Constraints (27)

Constraints (33) ~ (41)

$$Y_p \in \{0,1\} \quad \forall p \in P \quad (43)$$

The objective of the LBBD-MP is to maximize profit. Some of the relevant constraints in Model 2 are retained in the LBBD-MP, but some are omitted. Constraints (42) introduce an additional limitation to ensure that the combined area of allocated parts per build does not surpass the total area of the build.

4.1.2 Subproblems of the LBBD

By solving the subproblems of the LBBD, denoted as LBBD-SP_i, we can obtain optimal feasible part allocations to particular builds *i*. We define a set P_i of the parts that are initially allocated to build *i* in a solution of the master problem LBBD-MP, and a binary decision variable f_p which equals to 1 if part $p \in P_i$ is really allocated to build *i* in a solution of the subproblem LBBD-SP_i.

(LBBD-SP_i)

$$\max \sum_{p \in P_i} f_p$$

s. t.

Constraints (2) ~ (6)

$$x_j + L_j \leq x_k + BM \times (3 - o_{jk}^X - f_j - f_k) \quad \forall j, k \in P_i, j \neq k \quad (44)$$

$$o_{jk}^X + o_{kj}^X \leq 1 \quad \forall j, k \in P_i, j \neq k \quad (45)$$

$$y_j + w_j \leq y_k + BM \times (3 - o_{jk}^Y - f_j - f_k) \quad \forall j, k \in P_i, j \neq k \quad (46)$$

$$o_{jk}^Y + o_{kj}^Y \leq 1 \quad \forall j, k \in P_i, j \neq k \quad (47)$$

$$o_{jk}^X + o_{kj}^X + o_{jk}^Y + o_{kj}^Y - f_j - f_k \geq -1 \quad \forall j, k \in P_i, j \neq k \quad (48)$$

Constraints (21) and (23)

$$I_p^W, I_p^L, f_p \in \{0,1\} \quad \forall p \in P_i \quad (49)$$

The objective function is to maximize the number of parts to be included in build i . We determine the feasibility of the part allocation obtained from the LBBD-MP by comparing the objective function value of the subproblem with the number of parts in set P_i . The meaning of Constraints (44) ~ (48) have already been mentioned in Section 3.2. Constraints (21), (23) and (49) define the domains of decision variables.

4.1.3 Benders cuts of the LBBD

After solving LBBD-SP $_i$, if the value of the objective function for the subproblem is less than the number of parts in set P_i determined in the master problem LBBD-MP, it signifies that the part allocation defined by set P_i is infeasible. Consequently, we generate the following cuts and incorporate them into the master problem.

Benders cuts (V_1):

$$\sum_{p \in P_i} A_{mbp} \leq (|P_i| - 1) \quad \forall b \in B, m \in M \quad (50)$$

Equation (50) is a feasible cut, and its validity has been proven by Mao et al., (2024). Equation (50) indicate that in subsequent iterations of the LBBD method, the parts in set P_i cannot be packed in the same build.

4.1.4 The outline of the LBBD

The flow chart of the standard LBBD method is shown in Figure 3. The procedure begins by solving the master problem (LBBD-MP). If no feasible solution is found, the original problem is infeasible. If a feasible solution is obtained, the corresponding assignments of parts to builds and builds to machines are obtained. Subsequently, we evaluate the feasibility of part allocations within the LBBD-MP by solving the LBBD-SP $_i$ for all builds. If any subproblem is found to violate the 2D constraints, a set of Cuts (V_1) is generated. After checking the feasibility of all builds, we incorporate these cuts into the master problem. The above iterative process continues until all builds are feasible, indicating that the current solution satisfies both the master problem and the

2D constraints. At this point, the current solution is declared as the optimal solution to the original problem.

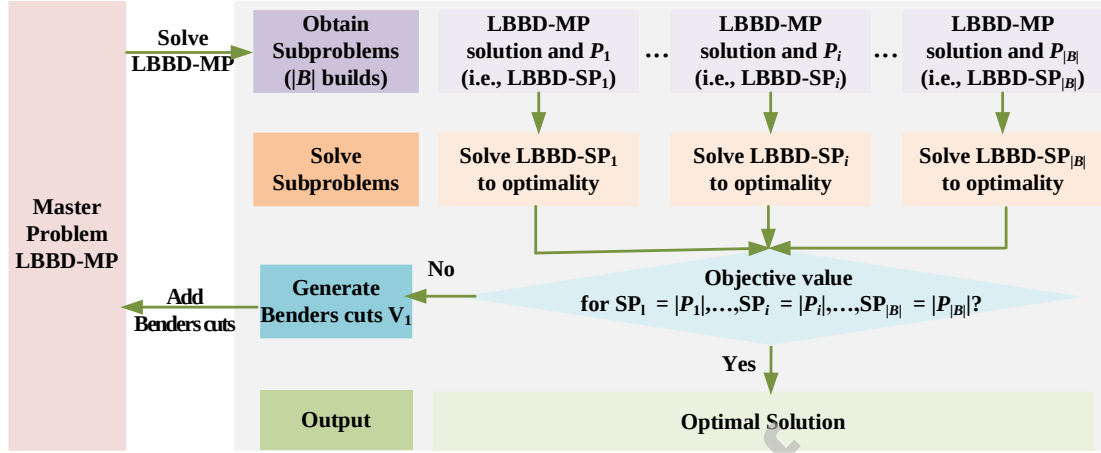


Figure 3. The flow chart of the standard LBB method

4.2 Nested logic-based Benders decomposition

Drawing upon the LBB method, we introduce a NLBB method. Specifically, we nest another layer of LBB to solve the LBB-MP. In other words, within the framework of NLBB method, we partition the original problem into a master problem (NLBB-MP) and two subordinate subproblems (NLBB-SP1_m and NLBB-SP2_b).

4.2.1 Master problem of the NLBB

In this section, we present the master problem of NLBB, comprising two pivotal decisions: the acceptance of parts and the assignment of the accepted parts to machines. In the previous problem description, decisions were centered around packing parts into the build before scheduling. An alternative way involves assigning parts to machines first, followed by packing these parts on each respective machine. We introduce several new decision variables, highlighting them in bold font to distinguish them from the ones used in LBB.

Decision variables and auxiliary variables determined by decision variables:

A_{pm} : 1 if part p is assigned to machine m , 0 otherwise

C_m : the total time of setup and height-dependent production on machine m

F_m : the minimum number of builds required on machine m

HH_m : the maximum height of parts on machine m

(NLBBD-MP)

$$\max \sum_{p \in P} U_p(1 - Y_p) - \alpha \cdot MS$$

s. t.

$$\sum_{m \in M} A_{pm} + Y_p = 1 \quad \forall p \in P \quad (51)$$

$$HH_m \geq BM \times (A_{pm} - 1) + h_p \quad \forall p \in P, m \in M \quad (52)$$

$$F_m \geq (\sum_{p \in P} A_{pm} \times l_p \times w_p) / (L \times W) \quad \forall m \in M \quad (53)$$

$$C_m \geq SET \times F_m + (F_m - 1) \times \min_{p \in P} (h_p) \times HT + HH_m \times HT \quad \forall m \in M \quad (54)$$

$$MS \geq C_m + \sum_{p \in P} A_{pm} \times v_p \times VT \quad \forall m \in M \quad (55)$$

$$HH_m, C_m \geq 0 \quad \forall m \in M \quad (56)$$

$$F_m \in \mathbb{Z}^+ \quad \forall m \in M \quad (57)$$

$$A_{pm} \in \{0,1\} \quad \forall m \in M, p \in P \quad (58)$$

$$Y_p \in \{0,1\} \quad \forall p \in P \quad (59)$$

The objective function is to maximize the profit. Constraints (51) select parts and assign the accepted parts to a machine. Constraints (52) compute the maximum height of the assigned parts on machine m . Constraints (53) calculate the minimum number of builds needed to pack the parts on machine m . Constraints (54) calculate the total time of setup and height-dependent production on machine m . Specifically, the term $SET \times F_m$ denotes the total setup time required on machine m , $HH_m \times HT$ represents the height-dependent processing time associated with the build that contains the tallest part assigned to machine m , and $(F_m - 1) \times \min_{p \in P} (h_p) \times HT$ accounts for the minimum total height-related processing time required for the remaining builds on machine m . Constraints (55) calculate the makespan. Constraints (56)-(59) define the domains of decision variables.

4.2.2 Subproblems of the NLBBD

After the NLBBD-MP is solved, the value of variables Y_p , A_{pm} are fixed. In other words, we know which parts are accepted and to which machines they are

assigned. Then, the NLBBD-SP1_m is solved to determine how to allocate the assigned parts into a build (excluding 2D constraints) to minimize the completion time on each machine m . Given that the volume-associated processing time remains constant regardless of how the part is allocated, we can disregard the volume-associated processing time in the objective function.

We define a set P_m of the parts that are assigned to machine m , i.e., $P_m = \{p | A_{pm} = 1, \forall p \in P\}$ and a new variable φ_m to calculate the total setup time and height-dependent processing time on machine m . Then, the NLBBD-SP1_m subproblem can be formulated as follows.

(NLBBD- SP1_m)

$$\min \varphi_m$$

s. t.

$$\sum_{b \in B} z_{pb} = 1 \quad \forall p \in P_m \quad (60)$$

$$\sum_{p \in P_m} z_{pb} \times l_p \times w_p \leq L \times W \quad \forall b \in B \quad (61)$$

$$\sum_{p \in P_m} z_{p(b+1)} \leq BM \times \sum_{p \in P_m} z_{pb} \quad \forall b \in B \quad (62)$$

$$H_b \geq BM \times (z_{pb} - 1) + h_p \quad \forall p \in P_m, b \in B, m \in M \quad (63)$$

$$bt_b \geq SET \times z_{pb} + HT \times H_b \quad \forall p \in P_m, b \in B \quad (64)$$

$$\varphi_m \geq \sum_{b \in B} bt_b \quad (65)$$

$$H_b, bt_b \geq 0 \quad \forall b \in B \quad (66)$$

$$z_{pb} \in \{0, 1\} \quad \forall p \in P_m, b \in B \quad (67)$$

The objective function is to minimize the completion time on a machine. Constraints (65) calculate the total setup time and height-dependent processing time. The other constraints have been explained in Section 3.2, which are not repetitively elaborated here.

After solving the NLBBD-SP1_m, we obtain the part allocations to machines. We then solve NLBBD-SP2_b, which is the same as LBBD-SP_i in LBBD method, to determine whether the part allocation to build b is feasible, i.e., satisfying 2D constraints. If it is infeasible, we generate the corresponding Benders cuts for the NLBBD-SP1_m. We define a set P_b of the parts that are allocated to build b , i.e., $P_b =$

$$\{p | z_{pb} = 1, \forall p \in P_m\}.$$

(NLBBD-SP2_b)

$$\max \sum_{p \in P_b} f_p$$

s. t.

Constraints (2) ~ (6)

$$x_j + L_j \leq x_k + BM \times (3 - o_{jk}^X - f_j - f_k) \quad \forall j, k \in P_b, j \neq k \quad (68)$$

$$o_{jk}^X + o_{kj}^X \leq 1 \quad \forall j, k \in P_b, j \neq k \quad (69)$$

$$y_j + w_j \leq y_k + BM \times (3 - o_{jk}^Y - f_j - f_k) \quad \forall j, k \in P_b, j \neq k \quad (70)$$

$$o_{jk}^Y + o_{kj}^Y \leq 1 \quad \forall j, k \in P_b, j \neq k \quad (71)$$

$$o_{jk}^X + o_{kj}^X + o_{jk}^Y + o_{kj}^Y - f_j - f_k \geq -1 \quad \forall j, k \in P_b, j \neq k \quad (72)$$

Constraints (21) and (23)

$$I_p^W, I_p^L, f_p \in \{0,1\} \quad \forall p \in P_b \quad (73)$$

4.2.3 Benders cuts of the NLBBD

In this section, we present a series of Benders cuts designed for the NLBBD method. Cuts (V_2) are generated by solving the NLBBD-SP2_b and added to NLBBD-SP1_m. Cuts (V_2) have the same meaning as Cuts (V_1) used in the LBBD approach, indicating that in the next iteration of the NLBBD-SP1_m, the parts in set P_b cannot be packed in the same build. The set P_b is obtained by solving the NLBBD-SP1_m, i.e., $P_b = \{p | z_{pb} = 1, \forall p \in P_m\}$.

Benders cuts (V_2):

$$\sum_{p \in P_b} z_{pb} \leq (|P_b| - 1) \quad \forall b \in B \quad (74)$$

When all feasible part allocations are achieved by solving the NLBBD-SP1_m, leading to the optimal objective function value ϕ_m^* . Cut (V_3) is then generated and incorporated into the NLBBD-MP.

Benders cuts (V_3):

$$C_m \geq \varphi_m^* (\sum_{p \in P_m} A_{pm} - |P_m| + 1) \quad \forall m \in M \quad (75)$$

Cuts (V_3) indicate that in subsequent iterations, if all parts in set P_m are still assigned to machine m , then the total setup time and height-dependent processing time on machine m is φ_m^* . Note that if there exists a part in the set P_m that is not assigned to the machine m in the following iterations, then Cuts (V_3) will become non-binding. Therefore, to tighten the boundaries of the NLBBD-MP and enhance the algorithm's performance, we introduce two additional cuts: Cuts (V_4) and (V_5). Both Cuts (V_4) and (V_5) are generated when the allocation of parts in set P_b is infeasible after solving the subproblems. The validity of these cuts is formally established in Appendix C.

Benders cuts (V_4):

$$F_m \geq 2 \times (\sum_{p \in P_b} A_{pm} - |P_b| + 1) \quad \forall m \in M \quad (76)$$

Benders cuts (V_5):

$$HM_m \geq \min_{p \in P_b} (h_p) \times (\sum_{p \in P_b} A_{pm} - |P_b| + 1) \quad \forall m \in M \quad (77)$$

Cuts (V_4) indicate that in subsequent iterations, if all parts in set P_b are still assigned to machine m , then at least two builds are needed on machine m to pack these parts. In Cuts (V_5), we introduce an auxiliary variable HM_m that represents the height of another build on machine m . In the NLBBD-MP, the total setup time and height-dependent processing time on machine m are calculated by Constraints (54). The height of a build on machine m is determined by the maximum height of all the parts assigned to machine m , while the height of the remaining builds is relaxed to the minimum height of all the parts. Thus, when we introduce the auxiliary variable HM_m , Constraints (54) are transformed into the following two type of constraints:

$$HM_m \geq \min_{p \in P} (h_p) \quad m \in M \quad (78)$$

$$C_m \geq SET \times F_m + (F_m - 2) \times \min_{p \in P} (h_p) \times VT + HM_m \times VT + HM_m \times HT \quad \forall m \in M \quad (79)$$

When no Cuts (V_5) are added to NLBBD-MP, Constraints (79) are equal to Constraints (54). Cuts (V_5) indicate that in subsequent iterations, if all parts in set P_b are still assigned to machine m , the height of another build on machine m is captured

by the minimum height of the parts in set P_b , i.e., $\min_{p \in P_b}(h_p)$.

4.2.4 The outline of the NLBBD

The flowchart of the standard NLBBD method is presented in Figure 4. This method aims to solve the integrated problem involving order acceptance, build formation and schedule construction, by embedding a two-level decomposition structure. The process begins with solving the master problem (NLBBD-MP), which determines the acceptance of parts and their assignment to machines. The solution of NLBBD-MP provides a part set $\mathbf{P}_m$ for each machine $m \in M$, which serves as the input to the first-level subproblems (NLBBD-SP1_m). These subproblems assign the accepted parts into builds, but without considering 2D constraints.

Subsequently, the second-level subproblems (NLBBD-SP2_b) are solved to verify the feasibility of the part layouts within each build under 2D constraints. If any infeasibility is detected, we incorporate Cuts (V_2) into NLBBD-SP1_m, Cuts (V_4) and (V_5) are generated and added to NLBBD-MP, and continue the iterative solving process. This ensures that the spatial constraints are respected in future iterations.

If all builds pass the feasibility check, i.e., all part allocations to builds obtained from NLBBD-SP1_m satisfy 2D constraints, the optimal solution for NLBBD-SP1_m is achieved. The optimal solution of the original problem is determined by comparing the objective value φ_m of the NLBBD-SP1_m with the value of $\mathbf{C}_m$ in the NLBBD-MP, $m = 1, \dots, |M|$. If a discrepancy remains, Cuts (V_3) are added to the master problem NLBBD-MP, and the iteration continues. Conversely, if for all machines the equality $\varphi_m = \mathbf{C}_m$ holds, it indicates that both feasibility and optimality conditions have been simultaneously satisfied across all levels of the decomposition. Hence, the current solution is guaranteed to be optimal for the original problem.

It is important to note that the NLBBD method is theoretically equivalent to the MIP formulation and the LBBD method in terms of the optimal objective value. Although the structure of intermediate solutions may differ due to decomposition granularity, the final solution produced by NLBBD lies within the feasible region of the MIP model and satisfies all problem constraints. Therefore, NLBBD not only preserves

solution optimality but also offers a more scalable and modular solving process by exploiting problem structure through nested decomposition.

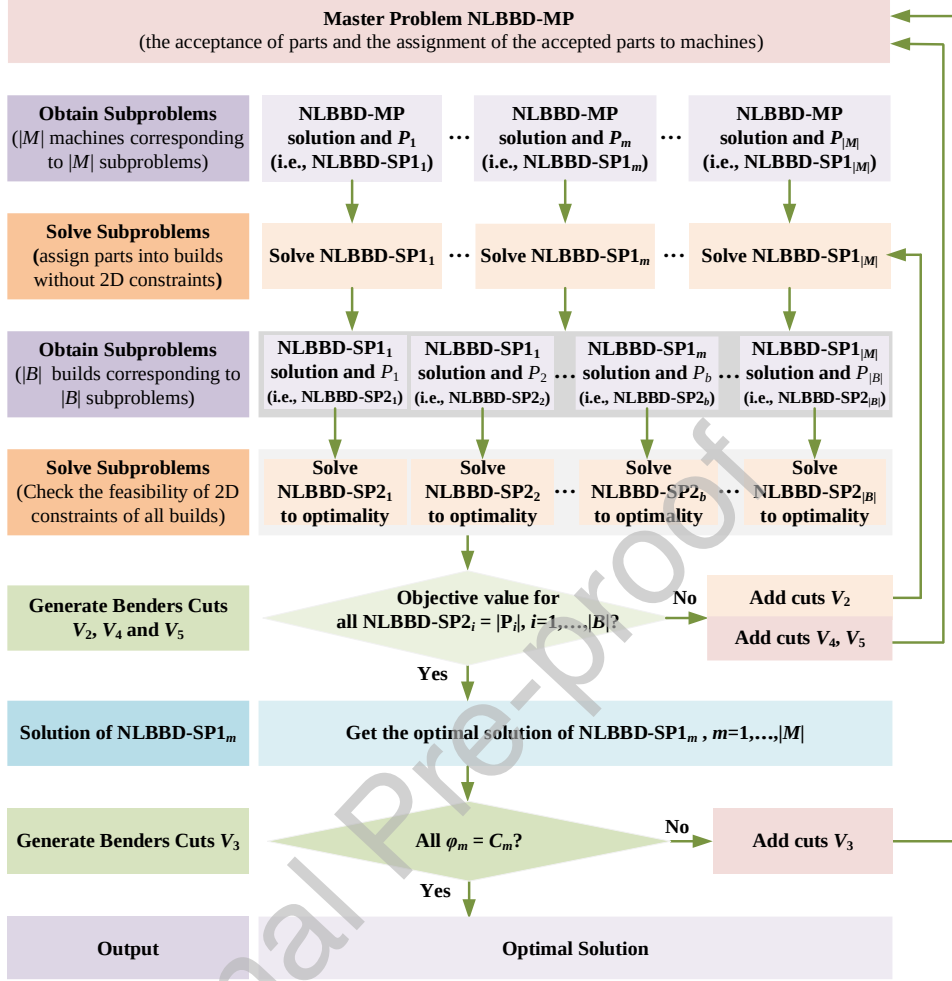


Figure 4. The flow chart of the standard NLBB method

4.3 Search strategies

In order to increase the efficiency of the Benders decomposition methods we employ three additional search strategies.

Strategy 1: Record feasible part allocations throughout the iterations (Nascimento et al., 2024).

We check the feasibility of part allocations in the LBB-MP and NLBB-SP_{1m} by solving the LBB-SP_i and NLBB-SP_{2b}, respectively. If an allocation is feasible, we add it to a list that records feasible part allocations during the iteration process. Feasible part allocations have a chance of being repeated in different iterations, therefore this strategy may allow to avoid double-checking feasibility of part allocations.

Strategy 2: Combine with the framework of branch-and-check.

Branch and check is a specific implementation of the Benders decomposition approach (Naderi and Roshanaei, 2020; Chen et al., 2023). The master problem is solved by the OR commercial solver, which uses a branch and cut algorithm for solving a mixed integer programming model. During branching, upon acquiring an integer feasible solution, we proceed to solve the subproblems associated with this solution. Benders cuts are then systematically generated and appended to each node of the master problem via a callback. Combining with the branch-and-check framework offers a significant advantage: the master problem must be solved only once throughout the entire iteration, resulting in a substantial reduction in solution time. This strategy is applied to both the LBBD and NLBBD algorithms in our study. For instance, in the NLBBD algorithm, we use a branch-and-cut approach to solve the master problem (NLBBD-MP) and obtain an integer feasible part-to-machine allocation. Based on this solution, the subproblems NLBBD-SP1_m and NLBBD-SP2_b are solved sequentially, and the generated Benders cuts are added to the remaining nodes of the branch-and-cut tree, thereby guiding the ongoing search.

Strategy 3: Start from a favorable initial solution in the NLBBD-MP.

In the Benders decomposition method, integrated with the branch-and-check framework, we aim to solve the subproblems for each obtained integer feasible solution and generate corresponding Benders cuts. However, initial integer feasible solutions often deviate significantly from the optimal solution, necessitating extensive time to iteratively approach optimality through cut generation. To address this, we propose strategy 3, which aims to provide the master problem with a high-quality initial solution. Specifically, we first solve the NLBBD-MP independently within a predefined time limit of 60 seconds, disregarding the subproblems. This step yields an initial feasible assignment of parts to machines in the original problem, regardless of whether the NLBBD-MP is solved to optimality within the time limit. Note that for any assignment of parts to machines, the subproblems can always determine valid build formations that satisfy the 2D constraints on each machine. Subsequently, the NLBBD algorithm is

executed under the branch-and-check framework, using the previously obtained solution as the starting point.

5. Computational study

In this section, the results of series of numerical experiments are presented. Computational experiments were conducted in order to evaluate the performance of the proposed models and algorithms. Some experiments were devoted to verify the validity of the cuts proposed in the Benders decomposition approach.

All the models and algorithms proposed in the paper are coded in Python linked with Gurobi 11.0.3. For each run, the optimization process terminates when the optimal solution is obtained or the predefined time limit of 3600 seconds is reached. All experiments were conducted on a server equipped with 2 virtual CPUs (Intel Xeon Gold 5218, 2.30 GHz base frequency), 64 GB of RAM, and running Windows 10.

5.1 Data generation

Without loss of generality, we assume that the data of all instances are integers. In what follows, when we use the term or the notation of uniform distribution, we refer to discrete uniform distribution on integers. For example, $U_p \sim U[10,40]$ means that the revenue of accepted part U_p follows the discrete uniform distribution on $\{10,11, \dots, 40\}$. The parameters setting used for instance generation are summarized in Table 2. To obtain a representative set of input instances of various characteristics, we divide the instances into two subsets according to instance sizes: (i) small-sized instances with $|P| \in \{10, 15, 20, 25, 30\}$, $|M| \in \{2, 3, 4\}$ and (ii) large-sized instances with $|P| \in \{40, 50, 60\}$, $|M| \in \{4, 5, 6\}$. For each combination of the numbers $|P|$ and $|M|$, four instances are randomly generated, leading to a total of $(5 \times 3 + 3 \times 3) \times 4 = 96$ instances.

The length, width and height of a part are generated from $U[5, 20]$. The width and length of a build are both set to 20 (Mao et al., 2024). The volume of a part is generated from $U\left[\frac{w_p \times l_p \times h_p}{3}, w_p \times l_p \times h_p\right]$, i.e., a range from one-third to full bounding box volume. The time spent per unit volume (VT), the time spent per powder-layering

(*HT*) and the setup time (*SET*) are set to 0.04, 0.7 and 2, respectively (Kucukkoc, 2019). Parameter α is set to 1.

Table 2. Summary of parameter settings

$ P \in \{10, 15, 20, 25, 30, 40, 50, 60\}$
$ M \in \{2, 3, 4, 5, 6\}$
$U_p: U[10, 40]$
$w_p: U[5, 20]$
$l_p: U[5, 20]$
$h_p: U[5, 20]$
$v_p: U\left[\frac{w_p \times l_p \times h_p}{3}, w_p \times l_p \times h_p\right]$
$W: W = 20$
$L: L = 20$
$SET: SET = 2$
$VT: VT = 0.7$
$HT: HT = 0.04$
$\alpha: \alpha = 1$

5.2 Computational results for small-sized instances

The 60 small-sized instances are solved by two proposed MIP models (Model 1 and Model 2) with Gurobi, the LBBB, and the NLBBB_V4V5. NLBBB_V4V5 denotes the NLBBB approach combined with all four Benders cuts (V_2), (V_3), (V_4), and (V_5). We report a relative gap for each instance, defined as follows

$$gap(\%) = \frac{UB-LB}{LB} \times 100 \quad (80)$$

where LB represents the objective value, while UB denotes the upper bound obtained by any solution method. Since all the master problems and subproblems outlined in Section 4 are solved using Gurobi solver with its branch and check framework, the UBs of four methods are also obtained through Gurobi.

The results of experiments are presented in Table 3, in which the first two columns indicate the number of parts ($|P|$) and the number of machines ($|M|$), respectively. For each ($|P|, |M|$) combination, four instances are solved. Table 3 reports the average objective (obj), average gap (gap/%), and average computation time in seconds (time/s)

across these instances. Detailed results for each individual run in Table 3 are provided in Table 6-10 in the Appendix D, and all test instances are available through the GitHub repository linked in the Data Availability section. The obj of each combination is determined as the average objective value of the best-found feasible solutions of its four instances within the time limit of 3600s. The value of gap is obtained by Gurobi according to Equation (80). Note that, in the obj column, the superscript in parentheses indicates the number of instances for which an optimal solution is found. For instance, (4, 1) denotes that optimal solutions were achieved in 1 out of 4 total runs. Moreover, in the row where $|P| = 10$ and $|M| = 4$, although the average obj of the four instances is 141.7, the superscript of obj for Model 1 is (4,2) rather than (4,4). This is because Gurobi confirms optimality in only two of the instances. In the remaining two runs, the solver terminates without verifying optimality within 3600s, as the optimality gap remains positive, i.e., the upper bound exceeds the lower bound in Equation (80).

Table 3. Computational results for small-sized instances

Instances		Model 1			Model 2			LBBD			NLBBD_V4V5		
$ P $	$ M $	Obj	Time/s	Gap/%	Obj	Time/s	Gap/%	Obj	Time/s	Gap/%	Obj	Time/s	Gap/%
10	2	86.3 ^(4,4)	8.7	0.0	86.3 ^(4,4)	5.1	0.0	86.3 ^(4,4)	3.6	0.0	86.3 ^(4,4)	1.8	0.0
	3	120.1 ^(4,4)	126.7	0.0	120.1 ^(4,4)	27.1	0.0	120.1 ^(4,4)	8.0	0.0	120.1 ^(4,4)	1.8	0.0
	4	141.7 ^(4,2)	2000.8	1.8	141.7 ^(4,4)	13.3	0.0	141.7 ^(4,4)	23.2	0.0	141.7 ^(4,4)	2.4	0.0
15	2	144.6 ^(4,4)	110.6	0.0	144.6 ^(4,4)	16.5	0.0	144.6 ^(4,4)	5.4	0.0	144.6 ^(4,4)	3.2	0.0
	3	184.1 ^(4,2)	2521.5	0.7	185.2 ^(4,4)	1229.0	0.0	185.2 ^(4,4)	455.8	0.0	185.2 ^(4,4)	15.4	0.0
	4	218.0 ^(4,0)	3600.0	22.3	222.5 ^(4,4)	1225.8	0.0	222.5 ^(4,3)	1402.8	0.5	222.5 ^(4,4)	63.4	0.0
20	2	134.6 ^(4,2)	2350.3	5.1	134.6 ^(4,3)	1146.8	0.4	134.6 ^(4,4)	60.1	0.0	134.6 ^(4,4)	24.5	0.0
	3	200.5 ^(4,0)	3600.0	24.0	201.6 ^(4,0)	3600.0	3.6	202.6 ^(4,0)	3600.0	1.5	202.6 ^(4,4)	127.8	0.0
	4	244.0 ^(4,0)	3600.0	43.9	252.0 ^(4,0)	3600.0	4.4	252.9 ^(4,0)	3600.0	2.9	253.2 ^(4,3)	2642.8	0.5
25	2	195.4 ^(4,0)	3600.0	51.1	199.1 ^(4,0)	3600.0	2.3	199.6 ^(4,4)	432.8	0.0	199.6 ^(4,4)	121.9	0.0
	3	266.9 ^(4,0)	3600.0	74.1	274.6 ^(4,0)	3600.0	3.9	276.2 ^(4,0)	3600.0	3.1	277.1 ^(4,1)	2716.8	0.9
	4	322.6 ^(4,0)	3600.0	51.0	329.4 ^(4,0)	3600.0	4.5	331.9 ^(4,0)	3600.0	3.0	333.0 ^(4,0)	3600.0	1.1
30	2	200.7 ^(4,0)	3600.0	158.5	218.2 ^(4,0)	3600.0	4.7	219.7 ^(4,1)	2869.8	1.5	219.7 ^(4,4)	412	0
	3	285.4 ^(4,0)	3600.0	103.8	303.6 ^(4,0)	3600.0	6.7	308.4 ^(4,0)	3600.0	3.9	307.9 ^(4,0)	3600.0	1.7
	4	341.0 ^(4,0)	3600.0	85.1	368.5 ^(4,0)	3600.0	6.6	372.4 ^(4,0)	3600.0	3.9	373.6 ^(4,0)	3600.0	2.3
Average		205.7 ^(60,18)	2634.6	41.4	212.1 ^(60,27)	2164.2	2.5	213.2 ^(60,32)	1790.7	1.4	213.4 ^(60,44)	1128.9	0.4

To further illustrate the algorithmic performance and enhance the interpretability of the numerical results, we provide a set of bar charts, as shown in Figure 5. Specifically, Figure 5(a) reports the total number of instances (out of 60) for which each method successfully identifies an optimal solution, offering insights into the effectiveness of each algorithm in reaching provably optimal results. Figure 5(b) presents the average objective values achieved by each solution method across all small-sized instances, thereby reflecting the overall solution quality. Figure 5(c) shows the average computation time required by each method, which serves as an indicator of algorithmic efficiency. Figure 5(d) illustrates the average optimality gap.

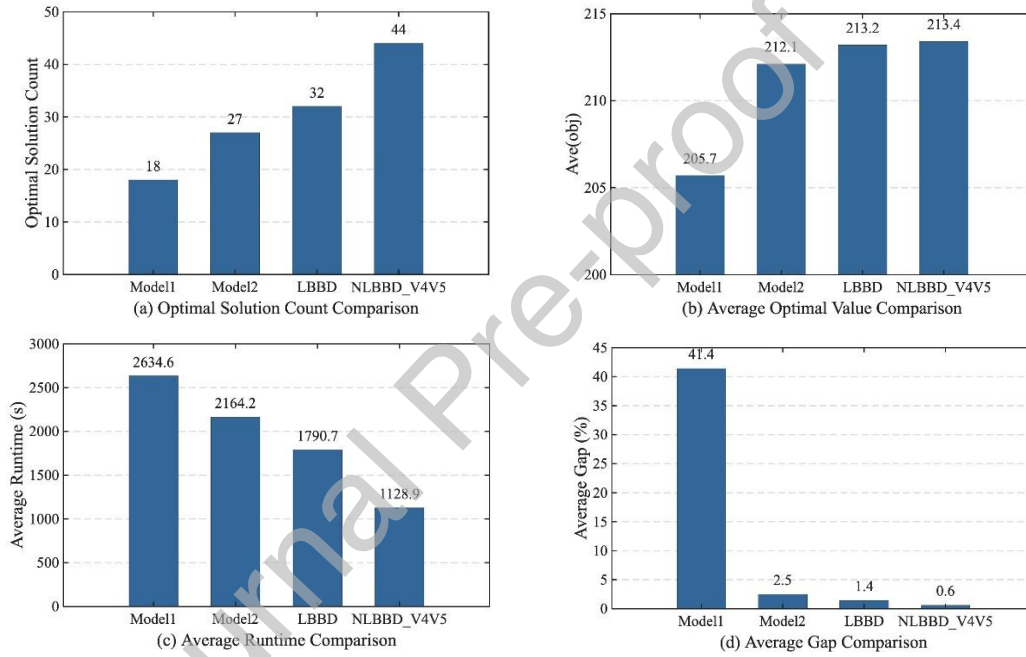


Figure 5. Algorithm performance comparison

Based on the results shown in Figure 5 and Table 4, the proposed NLBBD_V4V5 algorithm demonstrates the best overall performance across all evaluation metrics. It achieves the largest number of optimal solutions (44 out of 60), the highest average objective value (213.4), the lowest average computation time (1128.9s), and the smallest average optimality gap (0.6%), indicating both high solution quality and strong computational efficiency. The LBBD algorithm also performs well, achieving 32 optimal solutions with a low average gap of 1.4% and competitive runtime, positioning it as a robust alternative when the nested structure is not adopted. In contrast, Model 2, while superior to Model 1 in all aspects, i.e., producing more optimal solutions (27 vs.

18), a smaller gap (2.5% vs. 41.4%) and shorter runtime (2164.2s vs. 2634.6s), still falls behind the decomposition-based methods in both scalability and performance stability. Model 1 shows the weakest performance overall, with the lowest solution quality and largest computational burden, especially as the problem size increases. These results show the effectiveness of the proposed decomposition strategies, particularly the NLBBD, in tackling the OAPMSAM-2DPR problem by delivering high-quality solutions within reasonable time across a variety of instance sizes.

5.3 Computational results for large-sized instances

Based on the results gained from the small-sized instances, we additionally compared three most efficient approaches, i.e., Model 2, LBBD and NLBBD_V4V5, for large-sized instances. As in the small-sized experiments, four instances are solved for each $(|P|, |M|)$ combination. The average results are summarized in Table 4, while detailed results for each individual run are provided in Table 11-13 in the Appendix D.

Table 4. Computational results for large-sized instances

Instances		Model 2		LBBD		NLBBD_V4V5	
$ P $	$ M $	Obj	Gap/%	Obj	Gap/%	Obj	Gap/%
40	4	427.6	17.6	458.0	6.0	460.2	5.1
	5	478.7	20.0	529.8	5.9	535.0	5.1
	6	520.1	21.9	590.3	4.0	592.8	4.0
50	4	465.1	32.1	535.1	7.7	544.9	5.6
	5	439.1	66.0	608.9	9.6	636.1	5.3
	6	437.9	83.1	621.1	27.4	712.5	4.2
60	4	581.7	29.0	638.1	15.0	688.5	6.1
	5	405.1	160	667.3	27.5	801.8	5.1
	6	471.6	154	697.3	37.7	887.0	4.6
Average		469.6	64.8	593.9	15.6	651.0	5.0

Due to the complexity of the problem, none of the three methods could achieve an optimal solution within the time limit of 3600s for large-sized instances. As shown in Table 4, the performance comparison across large-sized instances reveals that

NLBBD_V4V5 consistently outperforms both Model 2 and LBBD in terms of solution quality and optimality gap. Specifically, NLBBD_V4V5 achieves the highest average objective value (651.0) and the lowest average optimality gap (5.0%), indicating its ability to deliver high-quality and near-optimal solutions. In contrast, Model 2 exhibits the weakest performance, with an average objective of only 469.6 and an average gap of 64.8%. LBBD performs moderately well, with an average objective value of 593.9 and a gap of 15.6%, but its effectiveness deteriorates as the problem size increases. Overall, the results demonstrate that NLBBD_V4V5 offers superior effectiveness, making it reliable for solving large-sized instances of the considered problem.

To further evaluate algorithmic stability, Figure 6 presents box plots of the gap values obtained by LBBD and NLBBD_V4V5 across different numbers of parts. The results illustrate that NLBBD_V4V5 consistently achieves small and stable gap values. In contrast, LBBD shows the larger variability in gap values between different instances, particularly noticeable with 60 parts. This confirms that NLBBD_V4V5 performs the best for large instances, consistently delivering satisfactory solutions with small gaps within 3600s, demonstrating robust stability.

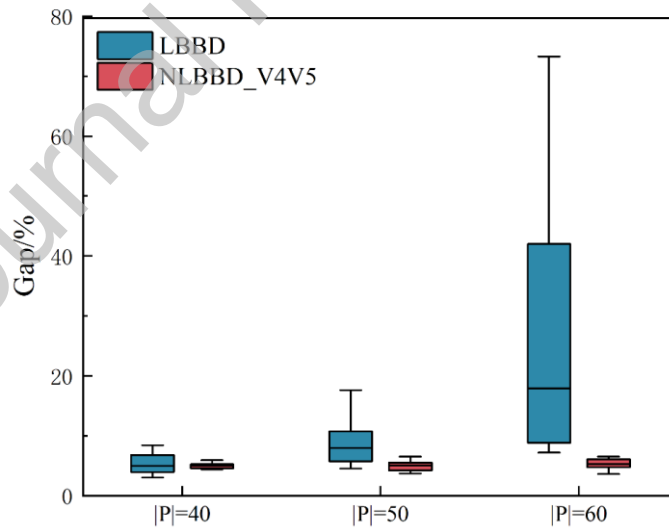


Figure 6. The box plots of the gaps for LBBD and NLBBD_V4V5

5.4 Performance of different Benders cuts

We investigate the influence of various Benders cuts on NLBBD's performance. Four variants of the NLBBD are compared, each incorporating a distinct combination

of cuts. The baseline variant, denoted as NLBBD, employs Cuts (V_2) and (V_3). The NLBBD_V4 incorporates Cuts (V_2), (V_3), and (V_4), while the NLBBD_V5 includes Cuts (V_2), (V_3), and (V_5). The NLBBD_V4V5 integrates Cuts (V_2), (V_3), (V_4), and (V_5).

To evaluate the effectiveness of each variant, we conducted computational experiments on 48 randomly generated instances with part quantities of 15, 20, 25, and 30. For instances with 15 and 20 parts, optimal solutions are consistently obtained across all variants, and average computation time is used as the performance metric. For instances with 25 and 30 parts, obtaining optimal solutions becomes intractable, so the evaluation metric is shifted to the average optimality gap. To ensure a consistent baseline, search strategies 1 and 2 are applied across all configurations. Search strategy 1 reduces redundant feasibility checks by recording valid part allocations, while search strategy 2 adopts a branch-and-check framework to generate cuts during branching. In contrast, search strategy 3 is an optional enhancement that supplies a promising initial solution to the master problem. Therefore, we isolate the effect of search strategy 3 under two settings: one with only search strategies 1 and 2, and the other with all three strategies enabled. The corresponding results are presented in Figure 7.

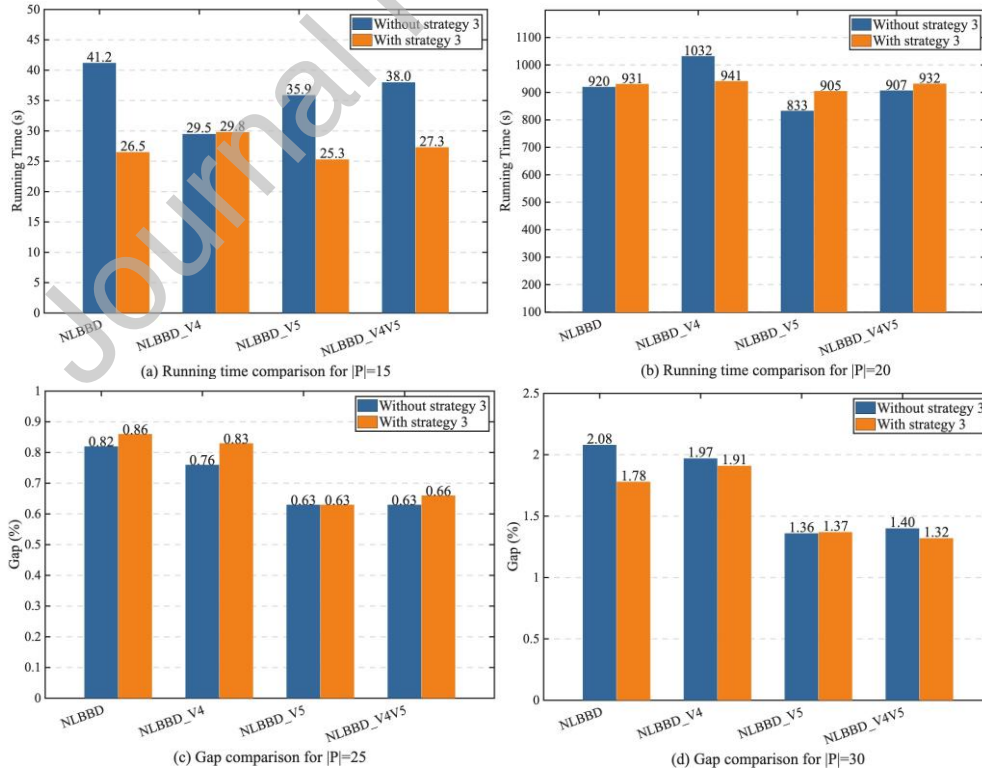


Figure 7. Comparison of different Benders cuts with and without strategy 3

Based on the results shown in Figure 7, we analyze the findings from two perspectives: the effect of different Benders cuts and the impact of incorporating Strategy 3. From the perspective of Benders cuts, the combination of Cuts (V_4) and (V_5) (i.e., NLBBD_V4V5) proves to be the most robust across problem scales. Although in small-sized instances (e.g., $|P|=15$), the runtime of NLBBD_V4V5 is comparable to or slightly higher than that of NLBBD_V4 or NLBBD_V5 individually, its performance becomes more advantageous as the problem size increases. Specifically, NLBBD_V4V5 achieves lower optimality gaps within the 3600s time limit for larger problem instances ($|P| = 25$ and 30), suggesting that the integration of both cuts provides a stronger cut structure and better guides the search in more complex scenarios. When comparing (V_4) and (V_5) individually, the results indicate that (V_5) generally performs better than (V_4) , implying that (V_5) contributes more to improving convergence. Nevertheless, both variants outperform the baseline NLBBD without any cut enhancement, confirming the effectiveness of incorporating cuts in the decomposition process. From the perspective of search strategy 3, its incorporation generally enhances computational efficiency across both small and large instances. For example, at $|P| = 15$, search strategy 3 clearly reduces the runtime for cut strategies, and at $|P| = 30$, it also slightly improves the optimality gaps. However, in two intermediate cases (i.e., $|P| = 20$ and 25), the performance of strategies with search strategy 3 is marginally less favorable in terms of gap compared to those without it. This suggests that while search strategy 3 provides a good initial solution and helps accelerate early convergence, its impact may vary slightly depending on the problem characteristics and solution space structure.

In summary, the results highlight the effectiveness of combining search strategy 3 with selected Benders cuts, particularly the integrated use of (V_4) and (V_5) . This leads to three findings: 1) In terms of scalability, although the joint use of (V_4) and (V_5) does not provide a distinct runtime advantage in small-sized instances, it substantially reduces the optimality gap as the problem size increases, indicating strong performance in large-scale scenarios; 2) Regarding the effectiveness of individual cuts,

(V_5) generally performs better than (V_4) in both solution quality and convergence, suggesting that (V_5) has a more influential role in guiding the search; 3) For the impact of search strategy 3, the incorporation of Strategy 3 tends to improve convergence speed and solution stability, though its benefits may be less pronounced for mid-sized instances, such as when $|P|$ equals 20 or 25.

6. Conclusions

This paper investigates an order acceptance and parallel AM machine scheduling problem considering 2D packing and part rotation, i.e., the OAPMSAM-2DPR problem. The objective is to maximize the profit, which is the revenue of accepting the parts decreased by the cost of AM production. To address the problem, we develop two MIP models for the problem varying in the idea of the decision variables. Then, based on the particular structure of the problem, we propose two exact Benders decomposition methods, i.e., LBBD and NLBBD. In addition, we design several Benders cuts and employ search strategies to further improve the performance of the proposed methods. The efficiency of all approaches was validated in the computational experiments performed for representative set of instances varying in their sizes.

The test results show that the proposed NLBBD performs the best among all methods. Further numerical experiments confirm the effectiveness of the designed additional Benders cuts incorporated into NLBBD. NLBBD achieves 50% more optimal solutions than MIP models and 35% more than LBBD for small-sized instances. Moreover, NLBBD delivers satisfactory solutions with small gaps for large-sized instances too, and it achieves better solutions with smaller gaps compared to MIP models and LBBD for such instances. While LBBD performs less effectively than NLBBD, it surpasses Model 2 by achieving more optimal solutions in small-sized instances and smaller gaps in large-sized instances.

Within our future research we would like improve the proposed approaches as well as explore new more realistic AM models. Firstly, we will focus on developing high-efficient solving algorithms for subproblems and designing new Benders cuts based on the problem features analysis. Secondly, incorporating additional critical parameters into the considered AM model, such as irregular shapes of produced parts, delay costs

and transportation expenses, could broaden its applicability. Thirdly, taking into account other optimization criteria such as flow time or due date related ones based on orders' delays would be an interesting research direction. Finally, this study demonstrates the effectiveness of the NLBBD framework for problems with hierarchical decisions and nested feasibility constraints. Extending it to other domains, such as geometric batch scheduling in semiconductor manufacturing or container loading with routing in logistics, would be an interesting research direction.

Acknowledgements

We thank the Editor and anonymous Referees for their valuable comments and constructive suggestions, which helped us significantly improve the quality of our article. This work was supported by the National Natural Science Foundation of China (52075259) and statutory funds of Poznan University of Technology (Poland).

Data availability

All benchmark instances used in this study are publicly available at the following GitHub repository: https://github.com/OurGroupNUAA/AM_Scheduling.git.

Appendix: Supplementary materials

Supplementary materials associated with this article can be found at https://github.com/OurGroupNUAA/ejor_appendix.git.

References

- Alicastro, M., Ferone, D., Festa, P., Fugaro, S., Pastore, T., 2021. A reinforcement learning iterated local search for makespan minimization in additive manufacturing machine scheduling problems. *Computers & Operations Research*, 131, 105272.
- Aloui, A., Hadj-Hamou, K., 2021. A heuristic approach for a scheduling problem in additive manufacturing under technological constraints. *Computers & Industrial Engineering*, 154, 107115.
- Attaran, M., 2017. The rise of 3-D printing: The advantages of additive manufacturing over traditional manufacturing. *Business Horizons*, 60, 677-688.
- Bartal, Y., Leonardi, S., Marchetti-Spaccamela, A., Sgall, J., Stougie, L., 2000. Multiprocessor scheduling with rejection. *SIAM Journal on Discrete Mathematics*, 13, 64-78.

- Benders, J.F., 1962. Partitioning procedures for solving mixed-variables programming problems. *Numerische Mathematik*, 4 (1), 238-252.
- Chen, J., Ma, W., Ye, X., Zhao, Z., 2023. Logic-based Benders decomposition for order acceptance and scheduling in distributed manufacturing. *Advanced Engineering Informatics*, 58, 102228.
- Chen, J., Ye, X., Ma, W., Xu, D., 2024. Logic-based Benders decomposition for order acceptance and scheduling on heterogeneous factories with carbon caps. *Computers & Operations Research*, 168, 106706.
- Côté, J.-F., Haouari, M., Iori, M., 2021. Combinatorial Benders decomposition for the two-dimensional bin packing problem. *INFORMS Journal on Computing*, 33, 963-978.
- Elçi, Ö., Hooker, J., 2022. Stochastic planning and scheduling with logic-based Benders decomposition. *INFORMS Journal on Computing*, 34, 2428-2442.
- Emami, S., Moslehi, G., Sabbagh, M., 2017. A Benders decomposition approach for order acceptance and scheduling problem: a robust optimization approach. *Computational and Applied Mathematics*, 36, 1471-1515.
- Emami, S., Sabbagh, M., Moslehi, G., 2016. A Lagrangian relaxation algorithm for order acceptance and scheduling problem: a globalised robust optimisation approach. *International Journal of Computer Integrated Manufacturing*, 29, 535-560.
- Fazel-Zarandi, M.M., Beck, J.C., 2012. Using logic-based Benders decomposition to solve the capacity- and distance-constrained plant location problem. *INFORMS Journal on Computing*, 24 (3), 387-398.
- Fera, M., Fruggiero, F., Lambiase, A., Macchiaroli, R., & Todisco, V., 2018. A modified genetic algorithm for time and cost optimization of an additive manufacturing single-machine scheduling. *International Journal of Industrial Engineering Computations*, 9 (4), 423-438.
- Geramipour, S., Moslehi, G., Reisi-Nafchi, M., 2017. Maximizing the profit in customer's order acceptance and scheduling problem with weighted tardiness penalty. *Journal of the Operational Research Society*, 68, 89-101.
- Hooker, J.N., 2007. Planning and scheduling by logic-based Benders decomposition. *Operations Research*, 55, 588-602.
- Hooker, J.N., Ottosson, G., 2003. Logic-based Benders decomposition. *Mathematical Programming*, 96, 33-60.
- Hu, K., Che, Y., Ng, T.S., Deng, J., 2024. Unrelated parallel batch processing machine scheduling with time requirements and two-dimensional packing constraints. *Computers & Operations Research*, 162, 106474.
- Hu, K., Che, Y., Zhang, Z., 2022. Scheduling unrelated additive manufacturing machines with practical constraints. *Computers & Operations Research*, 144, 105847.
- Kalantari, M., Rabbani, M., Ebadian, M., 2011. A decision support system for order acceptance/rejection in hybrid MTS/MTO production systems. *Applied Mathematical Modelling*, 35, 1363-1377.
- Kucukkoc, I., 2019. MILP models to minimise makespan in additive manufacturing

- machine scheduling problems. *Computers & Operations Research*, 105, 58-67.
- Lee, S.J., Kim, B.S., 2023. Two-stage meta-heuristic for part-packing and build-scheduling problem in parallel additive manufacturing. *Applied Soft Computing*, 136, 110132.
- Li, Q., Kucukkoc, I., Zhang, D.Z., 2017. Production planning in additive manufacturing and 3D printing. *Computers & Operations Research*, 83, 157-172.
- Li, X., Zhang, K., 2018. Single batch processing machine scheduling with two-dimensional bin packing constraints. *International Journal of Production Economics* 196, 113-121.
- Li, Y., Côté, J.-F., Coelho, L.C., Zhang, C., Zhang, S., 2023. Order assignment and scheduling under processing and distribution time uncertainty. *European Journal of Operational Research*, 305, 148-163.
- Liu, L., Wu, Z., Yu, Y., 2023. A branch-and-price algorithm to perform single-machine scheduling for additive manufacturing. *Journal of Management Science and Engineering*, 8, 273-286.
- Liu, P., Lu, X., 2020. New approximation algorithms for machine scheduling with rejection on single and parallel machine. *Journal of Combinatorial Optimization*, 40, 929-952.
- Mao, Z., Fu, E., Huang, D., Fang, K., Chen, L., 2024. Combinatorial Benders decomposition for single machine scheduling in additive manufacturing with two-dimensional packing constraints. *European Journal of Operational Research*, 317, 890-905.
- Naderi, B., Govindan, K., Soleimani, H., 2020. A Benders decomposition approach for a real case supply chain network design with capacity acquisition and transporter planning: wheat distribution network. *Annals of Operations Research*, 291, 685-705.
- Naderi, B., Roshanaei, V., 2020. Branch-Relax-and-Check: A tractable decomposition method for order acceptance and identical parallel machine scheduling. *European Journal of Operational Research*, 286, 811-827.
- Nascimento, P.J., Silva, C., Antunes, C.H., Moniz, S., 2024. Optimal decomposition approach for solving large nesting and scheduling problems of additive manufacturing systems. *European Journal of Operational Research*, 327, 92-110.
- Nobibon, F.T., Leus, R., 2011. Exact algorithms for a generalization of the order acceptance and scheduling problem in a single-machine environment. *Computers & Operations Research*, 38, 367-378.
- Pinedo M., 2012. *Scheduling*[M]. New York: Springer.
- Rom, W.O., Slotnick, S.A., 2009. Order acceptance using genetic algorithms. *Computers & Operations Research*, 36, 1758-1767.
- Roshanaei, V., Luong, C., Aleman, D.M., Urbach, D., 2017a. Propagating logic-based Benders' decomposition approaches for distributed operating room scheduling. *European Journal of Operational Research*, 257, 439-455.
- Roshanaei, V., Luong, C., Aleman, D.M., Urbach, D.R., 2017b. Collaborative operating room planning and scheduling. *INFORMS Journal on Computing*. 29 (3), 558-580.

- Roshanaei, V., Luong, C., Aleman, D.M., Urbach, D.R., 2020. Reformulation, linearization, and decomposition techniques for balanced distributed operating room scheduling. *Omega*, 93, 102043.
- Sang, Y.-W., Wang, J.-Q., Sterna, M., Blazewicz, J., 2025. Single machine scheduling with the total weighted late work and rejection cost. *Naval Research Logistics (NRL)*, 72 (2), 260-274.
- Shabtay, D., Gaspar, N., Kaspi, M., 2013. A survey on offline scheduling with rejection. *Journal of Scheduling*, 16, 3-28.
- Slotnick, S.A., 2011. Order acceptance and scheduling: A taxonomy and review. *European Journal of Operational Research*, 212 (1), 1-11.
- Slotnick, S.A., Morton, T.E., 1996. Selecting jobs for a heavily loaded shop with lateness penalties. *Computers & Operations Research*, 23, 131-140.
- Slotnick, S.A., Morton, T.E., 2007. Order acceptance with weighted tardiness. *Computers & Operations Research*, 34, 3029-3042.
- Tarhan, İ., Oğuz, C., 2021. Generalized order acceptance and scheduling problem with batch delivery: Models and metaheuristics. *Computers & Operations Research*, 134, 105414.
- Tran, T.T., Araujo, A., Beck, J.C., 2016. Decomposition methods for the parallel machine scheduling problem with setups. *INFORMS Journal on Computing*, 28 (1), 83-95.
- Wang, Z., Qi, Y., Cui, H., Zhang, J., 2019. A hybrid algorithm for order acceptance and scheduling problem in make-to-stock/make-to-order industries. *Computers & Industrial Engineering*, 127, 841-852.
- Xue, G., Wang, Z., 2023. Order acceptance and scheduling in the instant delivery system. *Computers & Industrial Engineering*, 182, 109395.
- Ying, K.-C., Fruggiero, F., Pourhejazy, P., Lee, B.-Y., 2022. Adjusted Iterated Greedy for the optimization of additive manufacturing scheduling problems. *Expert Systems with Applications*, 198, 116908.
- Zhang, L., Lu, L., Yuan, J., 2009. Single machine scheduling with release dates and rejection. *European Journal of Operational Research*, 198, 975-978.
- Zhou, S., Jia, Z., Jin, M., Du, N., 2022. Minimizing makespan on parallel batch processing machines with two-dimensional rectangular jobs. *Computers & Industrial Engineering* 169, 108167.
- Zipfel, B., Neufeld, J., Buscher, U., 2024. An iterated local search for customer order scheduling in additive manufacturing. *International Journal of Production Research* 62, 605-625.